



RTL Design Sherpa

DDR3 / LPDDR3 Family Controller Hardware Architecture Specification 0.3

June 13, 2026

Table of Contents

1 Scoria Has Index.....	8
2 Document Information.....	8
2.1 Related documents.....	8
2.2 Terminology.....	9
2.3 Revision History.....	9
2.4 How to read this book.....	10
3 Purpose and Scope.....	11
3.1 Purpose.....	11
3.2 Why a delta specification.....	11
3.3 What is in scope.....	11
3.4 What is out of scope.....	11
4 Document Conventions.....	12
4.1 Block marking.....	12
4.2 Signal and parameter naming.....	12
4.3 Timing values.....	13
4.4 Citations.....	13
5 Definitions and Acronyms.....	13
5.1 Interfaces.....	13
5.2 DRAM commands this document refers to.....	14
5.3 Timing parameters new to scoria.....	14
5.4 Terms used with a specific meaning here.....	15
6 Scope and Goals.....	15

6.1 What scoria is.....	15
6.2 Goals.....	15
6.3 Non-goals.....	16
6.4 What makes DDR3/LPDDR3 easier than it looks.....	16
6.5 What is genuinely harder.....	17
7 Block Diagram.....	17
7.1 The whole controller, marked against pumice.....	17
7.1.1 Figure 2.1: scoria block diagram, with inheritance marking.....	17
7.2 Reading the figure.....	18
7.3 What the figure deliberately omits.....	18
8 Module Hierarchy.....	19
8.1 Planned hierarchy.....	19
8.1.1 Top and macro tier.....	19
8.1.2 FUB tier, inherited unchanged.....	19
8.1.3 FUB tier, modified.....	20
8.1.4 FUB tier, new.....	20
8.2 Naming, and the package.....	20
9 What Is Inherited, and the Blocks That Change.....	21
9.1 The inheritance, stated once.....	21
9.2 The changes, in one table.....	21
9.3 Power-down: the one change small enough to cover here.....	22
10 Init Sequencer: RESET#, MR0-MR3, ZQ Calibration.....	23
10.1 The sequence is JEDEC's, verbatim.....	23

10.2 Three consequences for the sequencer.....	24
10.3 ZQ calibration.....	25
10.3.1 Why ZQ gets its own block.....	25
11 Write Leveling: the Interface, Not the Search.....	25
11.1 What write leveling is for.....	25
11.2 The decision: scoria provides the interface, firmware runs the search.....	26
11.3 What the block owes the system.....	26
11.4 The timing windows.....	27
11.5 The telemetry requirement, and why it is not optional.....	28
12 Refresh: DDR3 All-Bank and LPDDR3 Per-Bank.....	28
12.1 Marking: INHERITED, with a mode select.....	28
12.2 What pumice already has.....	28
12.3 LPDDR3 keeps the same mechanism.....	29
12.4 Refresh by memtype.....	29
12.5 Requirements.....	30
13 DFI v3.1 Master Interface, and the v2.1.1 Delta.....	30
13.1 The revision decision.....	30
13.2 What does NOT change, and a diff that lies about it.....	31
13.3 What changes and matters.....	31
13.3.1 Leveling: reworked outright.....	31
13.3.2 Low power: one request becomes two.....	32
13.3.3 Initialization and errors.....	32
13.4 What scoria does NOT implement.....	32

14 AXI4 Slave and APB CSR.....	33
14.1 AXI4 slave.....	33
14.2 APB CSR.....	33
14.3 The CDC on the CSR path, and why it is an async FIFO.....	33
15 scoria_pkg, and Build-Time vs Runtime.....	34
15.1 The package (decision D3).....	34
15.2 The rule: timings are runtime, geometry is build-time.....	34
15.3 New CSR groups.....	35
16 Verification Strategy, and the Open Questions.....	36
16.1 The target is the DFI boundary.....	36
16.2 What must be verified, beyond inheriting pumice's suites.....	36
16.3 Question resolutions (v0.2).....	37
16.3.1 Q2: ZQCS is a request, never a preemption.....	38
16.3.2 Q3: sample the prime DQ bit only.....	39
16.3.3 Q5: struck rather than deferred.....	39
16.4 What would make this a 1.0.....	39

List of Figures

Figure 2.1: scoria block diagram, with inheritance marking.....	17
---	----

List of Tables

Table 1: Table 0.1: Document information.....	8
Table 2: Table 0.2: Related documents.....	8
Table 3: Table 0.3: Revision history.....	9
Table 4: Table 1.1: Block markings.....	12
Table 5: Table 1.2: Commands, and which are new to this family member.....	14
Table 6: Table 1.3: New timing parameters.....	14
Table 7: Table 2.1: Top and macro tier.....	19
Table 8: Table 2.2: Modified FUBs.....	20
Table 9: Table 2.3: New FUBs.....	20
Table 10: Table 3.1: Every change, with its cause.....	21
Table 11: Table 3.2: DDR3 initialization requirements, from JESD79-3F.....	23
Table 12: Table 3.3: scoria_wrlvl_ifc responsibilities.....	26
Table 13: Table 3.4: Write-leveling timing windows.....	27
Table 14: Table 3.5: Refresh by memtype.....	29
Table 15: Table 4.1: The leveling interface, v2.1.1 against v3.1.....	31
Table 16: Table 4.2: DFI v3.1 features deliberately unimplemented.....	32
Table 17: Table 5.1: Build-time versus runtime.....	35
Table 18: Table 5.2: New CSR groups.....	35
Table 19: Table 6.1: Question resolutions.....	37

1 Scoria Has Index

Generated: 2026-09-30

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2 Document Information

Table 0.1: Document information

Field	Value
Title	scoria DDR3/LPDDR3 Family Controller — Hardware Architecture Specification
Version	0.3
Date	2026-09-29
Status	First edition, pre-RTL . Specifies the controller; does not describe an implementation
Scope	Controller architecture to the DFI v3.1 boundary, for DDR3 and LPDDR3
Not in scope	The PHY; board bring-up; the DDR4/LPDDR4 features DFI v3.1 also carries

2.1 Related documents

Table 0.2: Related documents

Document	Where	Relationship
scoria design requirements	../design-requirements.md	binding foundation — the spec deltas and settled decisions
pumice HAS	../.. /pumice-ddr2-lpddr2/docs/	the architecture scoria is

Document	Where	Relationship
pumice design requirements	pumice_has/ ../pumice-ddr2-lpddr2/docs/design-requirements.md	derived from its coding guidelines and enforcement rules apply here unchanged
JESD79-3F	outside the repo, cold storage	DDR3 device standard
JESD209-3C	outside the repo, cold storage	LPDDR3 device standard
DFI v3.1	dfi-specs/	the PHY boundary

2.2 Terminology

INHERITED / MODIFIED / NEW The marking every block in this document carries, relative to pumice. *Inherited* means the pumice block is used as-is, and its correctness argument transfers with it. *Modified* means a named, bounded change. *New* means no pumice counterpart exists.

nCK A DRAM clock cycle, the unit JESD79-3F states most command-spacing minimums in.

Prime DQ bit In write leveling, the DQ bit (or bits) on which the DRAM returns the leveling result.

Maintenance traffic Commands the controller must issue that no host requested — refresh and ZQ calibration. It competes with demand traffic for the command bus, which makes it a scheduling problem and not only a sequencing one.

2.3 Revision History

Table 0.3: Revision history

Version	Date	Change
0.3	2026-09-30	Q2 and D2 re-answered from a generated DDR3 LiteDRAM core instead of by analogy (Sean: “you might need to generate a ddr3 version of litedram”). ZQCS confirmed as request/grant sharing the refresher FSM; write

Version	Date	Change
0.2	2026-09-29	leveling confirmed as CSR-only with no state machine. Q1-Q5 resolved or deferred with conditions. refresh_ctrl corrected from MODIFIED to INHERITED: pumice already implements REFpb and the device owns the sequence, so Q5 was malformed and is struck.
0.1	2026-09-29	First edition. Written from the delta analysis, with decisions D1-D3 settled. No RTL exists; every block is specified, not described.

2.4 How to read this book

Chapter 1 fixes purpose and vocabulary. Chapter 2 is the shape of the controller, with one figure in which every block is colour-marked against pumice — read that figure and you know the size of this project. Chapter 3 is the only chapter with substantial new engineering: the four blocks DDR3 and LPDDR3 actually change. Chapter 4 is the DFI v3.1 boundary and what moved from v2.1.1. Chapter 5 is the package and parameters. Chapter 6 is how it will be verified, and the questions this edition cannot answer.

A caution particular to a pre-RTL specification. A document written after the fact can be incomplete; one written before can be *unbuildable*. Where this edition states a structure, it is inherited from a controller that is built and measured. Where it states a timing, it is cited. Where neither is true, it says so in Chapter 6 rather than guessing.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

3 Purpose and Scope

3.1 Purpose

scoria is a unified, parameterized memory controller for DDR3 SDRAM and LPDDR3 SDRAM, presenting an AXI4 slave to the host and a DFI v3.1 master to the PHY. It is the third member of a family: pumice covers DDR2/LPDDR2 and is built, measured and running on hardware; a DDR4/LPDDR4 controller is planned.

This document specifies scoria's architecture. Its purpose is to be *locked* before RTL begins, so that the implementation has a fixed target and the PRD — which explicitly waits on a locked HAS — can be written.

3.2 Why a delta specification

scoria is not a fresh design. The controller architecture it uses is pumice's, and that architecture has been through board bring-up, a read-path rebuild that took reads from 291.7 to 571.3 MB/s, and a formal campaign that found two real bugs. Re-deriving it would discard that evidence.

So this specification is organised as a delta. Every block is marked **INHERITED**, **MODIFIED** or **NEW**, and the marking carries an obligation:

- **INHERITED** means the pumice block is used unchanged, and its verification evidence transfers. If a block is marked inherited and then changed during implementation, the marking is wrong and this document must be corrected — not the marking quietly dropped.
- **MODIFIED** means a bounded change with a named cause, always a clause of JESD79-3F, JESD209-3C or DFI v3.1.
- **NEW** means no pumice counterpart. There are two, and both are small.

3.3 What is in scope

The controller between the AXI4 slave port and the DFI v3.1 master port: the front end, the scheduler, bank and global timing enforcement, refresh, mode registers and initialization, ZQ calibration, the write-leveling interface, the data paths, and the CSR block.

3.4 What is out of scope

The PHY. DFI is the boundary, as it is for pumice, and for the same reason: IOB serdes, bitflip and delay-line calibration are FPGA- or process-specific and do not belong in a portable controller. This is a deliberate architectural commitment, not an omission.

The write-leveling search. DDR3 adds write leveling, and scoria provides the interface to perform it — the DFI handshake, the MR1 path, the timing windows and the telemetry. The search algorithm itself runs in firmware. This was settled as decision D2 and is argued in Chapter 3.

The DDR4/LPDDR4 half of DFI v3.1. Targeting v3.1 brings signals scoria has no use for: `dfi_act_n`, bank groups, chip ID, CA parity and `dfi_alert_n`, Data Bus Inversion, and CA training. They are named here so their absence is a decision on the record rather than an oversight.

Board bring-up. Verification is against a DFI bus functional model in simulation. The board target is deliberately deferred.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 Document Conventions

4.1 Block marking

Every block carries exactly one marking, and the block diagram colours match:

Table 1.1: Block markings

Marking	Colour in Figure 2.1	Meaning
INHERITED	green	pumice's block, used as-is; its verification evidence transfers
MODIFIED	amber	a bounded change with a cited cause
NEW	red	no pumice counterpart

4.2 Signal and parameter naming

Inherited from the pumice design requirements and not restated here: module naming, UPPER_CASE parameters, `r_/w_` internal prefixes, and the per-area clock and reset names. Two points bear repeating because they are load-bearing:

- **Clock and reset names are per area.** In the component tree the controller uses `aclk/aresetn`. An `.i_rst_n(...)` connection against an `rtl/common` module will not elaborate, because those use `clk/rst_n`.
- **No assertions in the RTL.** Properties live in external `formal/` blocks. This is a tool-compatibility rule, not a style preference.

4.3 Timing values

Every enforced timing is a runtime CSR, not a parameter. This is inherited from pumice and was learned there: a timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized.

Where this document states a numeric timing it is cited to a clause, and the unit is given. `nCK` means DRAM clock cycles. Values that vary by speed bin are named but not tabulated here — they belong to the CSR derivation.

Note: JESD79-3F states some minimums without a maximum, most notably `tWLMRD`, whose maximum it declares controller-dependent. Where the spec declines to bound something, this document says so rather than inventing a bound.

4.4 Citations

A claim traceable to a standard is cited as JESD79-3F (DDR3), JESD209-3C (LPDDR3), or DFI v3.1 with a section number. Those PDFs are **not in the repository** and must not be added to it — they are not redistributable. They live in the operator's cold storage; `dfi-specs/ddr3/index.md` records where and why.

A claim inherited from pumice cites the pumice HAS or its design requirements, both of which are in the repository.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

5 Definitions and Acronyms

5.1 Interfaces

AXI4 The host-side slave interface. Inherited from pumice unchanged, including the width-gearing converters that let the host bus width differ from the DRAM's.

DFI DDR PHY Interface. scoria targets **v3.1** (decision D1). The boundary between controller and PHY, and the same boundary the DV repo's bus functional model drives in simulation.

APB The CSR access interface, through a PeakRDL-generated register block.

5.2 DRAM commands this document refers to

Table 1.2: Commands, and which are new to this family member

Command	Present in DDR2?	Note
ACT, RD, WR, PRE, REF	yes	inherited handling
PREA	as PRE with A10	DDR3 gives precharge-all its own encoding
MRS	yes	but DDR3 has MR0-MR3 where DDR2 had MR0-MR2 plus EMRS3
ZQCL / ZQCS	no	ZQ calibration, long and short. New in DDR3
RESET	no	DDR3 adds a hardware RESET# pin
SRE / SRX	yes	self-refresh entry and exit
REFpb	LPDDR2 has it	LPDDR3 keeps per-bank refresh

5.3 Timing parameters new to scoria

Table 1.3: New timing parameters

Parameter	Governs
tZQinit	ZQ calibration during initialization
tZQoper	a long ZQ calibration outside initialization
tZQCS	the short, periodic ZQ calibration
tWLMRD	delay from entering write-leveling mode to the first DQS pulse
tWLDQSEN	delay before the controller may drive DQS, by which point the DRAM has

Parameter	Governs
tWLO	applied ODT write-leveling output delay: when the DRAM returns the result
tWLOE	DQ output uncertainty, allowing mismatch across DQ bits

All seven are runtime CSRs. tZQinit, tZQoper and tZQCS also gate command issue alongside tMRD, tMOD and tRFC (JESD79-3F).

5.4 Terms used with a specific meaning here

Maintenance traffic Commands no host requested: refresh and ZQ calibration. Because it competes for the command bus, it is the scheduler's concern and not only the sequencer's.

Demand traffic Reads and writes originating from the AXI4 port.

Prime DQ bit The DQ bit (or bits) carrying the write-leveling result back from the DRAM.

Firmware-driven Performed by software over the CSR interface rather than by a hardware state machine. Both PHY leveling on pumice's board and scoria's write-leveling search are firmware-driven, deliberately.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

6 Scope and Goals

6.1 What scoria is

A memory controller presenting an AXI4 slave to the host and a DFI v3.1 master to the PHY, supporting DDR3 SDRAM and LPDDR3 SDRAM from one parameterized design.

6.2 Goals

1. **Reuse the pumice architecture, and its evidence with it.** The front end, scheduler, timing enforcement and data paths are inherited. Reuse here is not laziness; those blocks carry board measurements and formal proofs that a rewrite would discard.

2. **Change only what the standards force.** Four blocks change and two are new. Chapter 3 is that list, and it is short on purpose.
3. **Keep the DFI boundary clean.** No PHY responsibilities migrate into the controller — including the write-leveling *search*, which DDR3 makes tempting and decision D2 rules out.
4. **Every enforced timing a runtime CSR.** Inherited from pumice, and the precondition for characterization.
5. **Verifiable before hardware.** The DFI bus functional model is the target, so the controller can be wrong in simulation where it is cheap.

6.3 Non-goals

Performance parity with a vendor controller at DDR3 speeds. scoria’s design point is architectural clarity and measurability. pumice reaches roughly 95% of theoretical peak in both directions on its board, and the same structure should carry, but no throughput target is set in this edition.

The DDR4 feature set. DFI v3.1 carries it; scoria does not implement it.

Multi-rank beyond pumice’s parameterization. NUM_RANKS is inherited. As with pumice, the practical limit is the board, not the controller.

6.4 What makes DDR3/LPDDR3 easier than it looks

Three things reduce the work substantially, and it is worth stating them so the project is not over-scoped:

- **LPDDR3 is not a protocol break.** JESD209-3C keeps the 10-bit double-data-rate CA bus that LPDDR2 uses, so the LPDDR3 side is timings and mode registers rather than a new command encoding.
- **pumice was written expecting this.** Its row field is already padded to 18 bits with the comment “DDR3 forward-compat”, and memtype_e is an enum rather than a boolean — though a one-bit one, which is why decision D3 gives scoria its own package.
- **DFI v3.1 does not disturb the datapath.** Its multi-phase signalling generalises pumice’s explicit _p0 . . p3 to _pN, and the 1:4 frequency ratio is still defined. DFI_RATE = 4 carries over untouched.

6.5 What is genuinely harder

- **Write leveling.** New to this family member, and it spans the controller, the DRAM's MR1 and the PHY. The interface is specified here; the search is not.
- **ZQ calibration as maintenance traffic.** Periodic ZQCS has to be interleaved with demand traffic, which makes it a scheduling problem.
- **A four-register mode-register set.** MR0-MR3 replaces MR0-MR2 plus EMRS3, and the programming order is JEDEC's. pumice learned that the hard way: its init order was corrected to JEDEC's after an EMRS3-first sequence, which was benign but wrong.

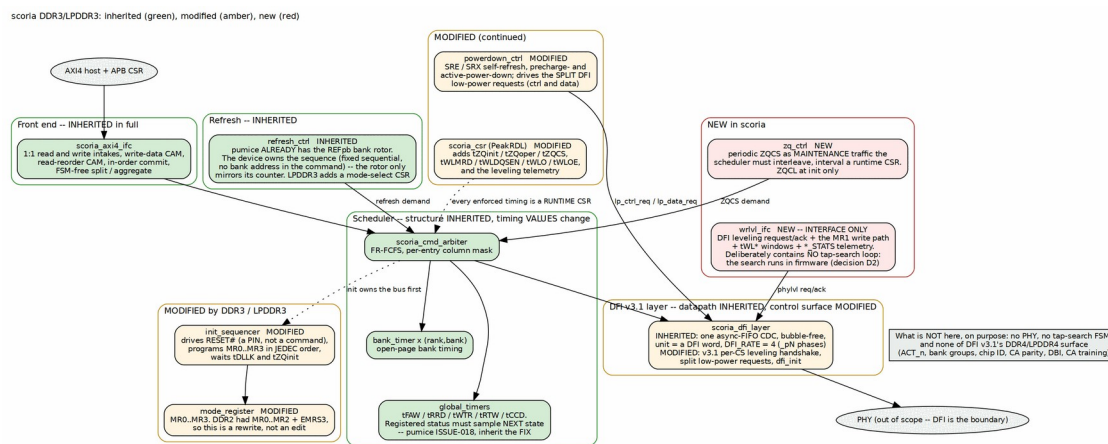
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

7 Block Diagram

7.1 The whole controller, marked against pumice

Read this figure first. The colour of each block tells you the size of this project: green blocks are pumice's, used as-is; amber blocks change in named, bounded ways; red blocks are new, and there are two.

7.1.1 Figure 2.1: scoria block diagram, with inheritance marking



scoria DDR3/LPDDR3: inherited (green), modified (amber), new (red)

Source: [01_block_diagram.dot](#)

7.2 Reading the figure

The front end is entirely green. The AXI4 interface — 1:1 read and write intakes, the write-data CAM, the read-reorder CAM, in-order commit, and the FSM-free split/aggregate — is untouched by the move to DDR3. Nothing about DDR3 or LPDDR3 reaches the host side.

The scheduler is green in structure, and its timing values change. The FR-FCFS arbiter, the per-(rank,bank) bank timers and the global timers are inherited. What changes is the numbers they enforce, and those are runtime CSRs rather than structure.

Important: one inherited detail must be inherited *including its fix*. pumice's `global_timers` published readiness flags one cycle late — registered status sampled the current state instead of the next — so the flags alone permitted violations of tCCD and tRTW. That was pumice ISSUE-018, and the fix was to compute one next-state function and feed both the counter and its status flop from it. scoria inherits the fixed form. A fresh implementation of the same block from the same description would reintroduce the bug.

The amber blocks are the four the standards change: the init sequencer, the mode-register block, refresh, and power-down — plus the CSR block, which grows to hold the new timings.

The red blocks are the two new ones, and both are deliberately small: a ZQ controller that issues periodic ZQCS as maintenance traffic, and a write-leveling interface that contains no search loop.

7.3 What the figure deliberately omits

The PHY, because DFI is the boundary. A tap-search state machine, because decision D2 puts the search in firmware. And DFI v3.1's DDR4/LPDDR4 signals, because scoria does not implement them. The note in the figure says so, so that their absence reads as a decision rather than an omission.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 Module Hierarchy

8.1 Planned hierarchy

scoria mirrors pumice's three-tier shape — FUBs under macros under a top — with the `scoria_` prefix replacing `pumice_`. The table below is the planned module list with its marking; it is a specification, and the RTL is expected to match it or this document to be corrected.

8.1.1 Top and macro tier

Table 2.1: Top and macro tier

scoria module	pumice counterpart	Marking
<code>scoria_top</code>	<code>pumice_top</code>	INHERITED (structure)
<code>scoria_top_geared</code>	<code>pumice_top_geared</code>	INHERITED — the host/DRAM width-gearing wrapper
<code>scoria_core</code>	<code>pumice_core</code>	INHERITED
<code>scoria_axi4_ifc</code>	<code>pumice_axi4_ifc</code>	INHERITED
<code>scoria_mem_cmd_scheduler</code>	<code>pumice_mem_cmd_scheduler</code>	MODIFIED — must admit ZQ maintenance demand
<code>scoria_dfi_layer</code>	<code>pumice_dfi_layer</code>	MODIFIED — v3.1 control surface

8.1.2 FUB tier, inherited unchanged

`addr_mapper`, `bank_timer`, `scoria_bank_timers`, `global_timers` (in its fixed form — see Chapter 2.2), `scoria_cmd_arbiter`, `scoria_page_policy`, `scoria_rd_cmd_cam`, `scoria_wr_data_cam`, `scoria_rd_intake`, `scoria_wr_intake`, `scoria_wr_splitter`, `scoria_rd_return_ring`, `scoria_axi_burst_chopper`, `scoria_dfi_cdc`, `scoria_dfi_cmd_path`, `scoria_dfi_rd_aligner`, `scoria_dfi_wr_serializer`, `scoria_cmd_history_checker`, and **`refresh_ctrl`** — which already carries the REFpb bank rotor for LPDDR2 and needs only a mode-select CSR for LPDDR3.

That is 19 of the 24 FUBs carried over with no functional change. The command history checker is worth naming explicitly: it is the verification-side block that independently re-derives JEDEC spacing from the issued command stream, and it must grow DDR3's parameters, but its mechanism is inherited.

8.1.3 FUB tier, modified

Table 2.2: Modified FUBs

Module	Why it changes	Detail
init_sequencer	DDR3 adds a RESET# pin and a four-register MR set	Ch 3.2
mode_register	MR0-MR3 replaces MR0-MR2 plus EMRS3	Ch 3.2
powerdown_ctrl	self-refresh entry/exit against the <i>split</i> DFI low-power requests	Ch 3.1
dfi_cmd_formatter	new command encodings: ZQCL, ZQCS, PREA	Ch 3.2
dfi_signal_pack	v3.1 signal set	Ch 4.1

8.1.4 FUB tier, new

Table 2.3: New FUBs

Module	Purpose
scoria_zq_ctrl	issues ZQCS periodically as maintenance traffic; interval a runtime CSR. ZQCL is init-only and belongs to the init sequencer
scoria_wrlvl_ifc	the write-leveling interface: DFI leveling handshake, MR1 write path, tWL* window enforcement, *_STATS telemetry. No search loop

8.2 Naming, and the package

Per decision D3, scoria gets its own `scoria_pkg` with a one-bit memtype enum covering {DDR3, LPDDR3}. It does not extend `pumice_pkg`, whose `memtype_e` is one bit wide and already spent on {DDR2, LPDDR2} — widening it would change a CSR in a design that is measured and shipping, for a controller with no RTL.

When the DDR4/LPDDR4 controller begins, a shared `mem_ctrl_pkg` with a two-bit `memtype` becomes worth the migration, and both existing controllers move to it together. That is the right time because the cost is paid once with three members to amortise it, rather than twice.

Note: this means `scoria_pkg` and `pumice_pkg` will contain near-identical type definitions for a period. That duplication is deliberate and time-boxed, and it is recorded here so it is not mistaken for an oversight and “fixed” by someone widening `pumice`’s CSR.

9 What Is Inherited, and the Blocks That Change

9.1 The inheritance, stated once

Nineteen of `pumice`’s twenty-four FUBs carry over with no functional change, as do the top, core, AXI4 macro and the DFI datapath. Chapter 2.3 lists them. This chapter covers only what changes, and the list is short by design.

The inheritance is an argument, not a convenience. Those blocks have been through board bring-up, a read-path rebuild that took reads from 291.7 to 571.3 MB/s, and a formal campaign across nine modules. Re-deriving them would discard that evidence and reintroduce the bugs it caught.

Important — inherit `global_timers` in its FIXED form. `pumice`’s version published readiness one cycle late: the registered status flop sampled the current counter state rather than the next, so the flags alone permitted tCCD and tRTW violations. The fix computes one next-state function and feeds both the counter and its status flop from it. A fresh implementation written from a behavioural description of the same block would reintroduce the defect, because the description does not mention the flop. Copy the module.

9.2 The changes, in one table

Table 3.1: Every change, with its cause

Block	Marking	Cause	Where
<code>init_sequencer</code>	MODIFIED	RESET# is a pin; MR order is MR2-MR3-MR1-MR0; ZQCL closes init	3.2

Block	Marking	Cause	Where
mode_register	MODIFIED	MR0-MR3 replaces MR0-MR2 plus EMRS3	3.2
dfi_cmd_formatter	MODIFIED	new encodings: ZQCL, ZQCS, PREA	3.2
scoria_zq_ctrl	NEW	periodic ZQCS is maintenance traffic	3.2
scoria_wrlvl_ifc	NEW	DDR3 adds write leveling	3.3
refresh_ctrl	INHERITED	pumice already implements REFpb; LPDDR3 uses the same device-fixed mechanism. Only a mode-select CSR is added	3.4
powerdown_ctrl	MODIFIED	self-refresh against the split DFI low-power requests	below
scoria_mem_cmd_scheduler	MODIFIED	must admit ZQ demand alongside refresh	3.2
scoria_csr	MODIFIED	new timing registers and leveling telemetry	Ch 5

9.3 Power-down: the one change small enough to cover here

DDR3 and LPDDR3 both offer precharge-power-down, active-power-down and self-refresh (SRE / SRX). pumice's powerdown_ctrl already implements idle-timer-driven entry and exit; what changes is the interface beneath it.

DFI v2.1.1 had a single `dfi_lp_req`. DFI v3.1 **splits it** into `dfi_lp_ctrl_req` and `dfi_lp_data_req`, letting the controller request low-power state for the command path and the data path independently. `powerdown_ctrl` therefore drives two requests where it drove one, and the distinction is real: a controller can idle the data path while keeping the command path alive, which is what active-power-down wants.

Note: the reverse asymmetry — bringing one back up without the other — is where a naive implementation will deadlock. The requirement in this edition is that entry and exit are symmetric per path and that the CSR exposes which paths are currently down. The exact handshake ordering is an open question in Chapter 6, because it depends on PHY behaviour this specification cannot fix.

10 Init Sequencer: RESET#, MR0-MR3, ZQ Calibration

10.1 The sequence is JEDEC's, verbatim

JESD79-3F specifies DDR3 power-up initialization as a numbered sequence. It is reproduced here as *requirements on the sequencer*, in the spec's order, because the order is not ours to choose:

Table 3.2: DDR3 initialization requirements, from JESD79-3F

Step	Requirement
1	Power ramp, with the spec's VDD / VDDQ / VTT / Vref ordering constraints. Board-level, not controller-level
2	After RESET# is de-asserted, wait 500 us before CKE becomes active. The DRAM initializes internally during this, independent of external clocks
3	Clocks stable for at least 10 ns or 5 tCK, whichever is larger , before CKE goes active. A NOP or Deselect must be registered before CKE goes high,

Step	Requirement
	meeting tIS
4	CKE must be continuously registered high from that point until initialization finishes — including expiry of tDLLK and tZQinit
5	Wait tXPR = max(tXS, 5 x tCK) after CKE high before the first MRS
6	MRS to MR2
7	MRS to MR3
8	MRS to MR1 , with DLL enabled
9	MRS to MR0 , with DLL reset
10	ZQCL to start ZQ calibration
11	Wait for both tDLLK and tZQinit
12	Ready for normal operation

10.2 Three consequences for the sequencer

RESET# is a pin, not a command. This is the structural change. pumice’s init sequencer issues commands; scoria’s must additionally drive a reset output and hold it through a timed window. That output has to reach the top level and the PHY, and it is the first thing in the sequence rather than something the command path can express.

The MR order is MR2, MR3, MR1, MR0 — and it is not sorted. Steps 6 through 9 are explicit, and MR1 carries “DLL enable” while MR0 carries “DLL reset”, so the order encodes a dependency rather than a preference.

Note: pumice hit precisely this. Its init programmed EMRS3 before EMRS2; the error was benign in practice but wrong, and it was corrected to JEDEC’s order. DDR3’s MR2-MR3-MR1-MR0 is the same pattern extended to a four-register set, and the same rule applies: the sequence is a citation, not a design choice.

ODT is constrained during init. The DRAM holds on-die termination in high-impedance while RESET# is asserted and until CKE is registered high; thereafter ODT must be held statically — and **statically LOW if RTT_NOM is to be enabled in MR1** — until initialization completes. The sequencer therefore owns ODT during init, not just during traffic.

10.3 ZQ calibration

ZQCL closes initialization (step 10) and t_{ZQinit} gates readiness (step 11). Outside init, DDR3 offers ZQCS, the short calibration, and t_{ZQoper} for a long one. All three gate command issue alongside $tMRD$, $tMOD$ and $tRFC$.

10.3.1 Why ZQ gets its own block

ZQCL at init belongs to the sequencer. Periodic ZQCS does not, because it is **maintenance traffic**: the controller must issue it on an interval no host asked for, competing with demand traffic for the command bus. That is the same shape as refresh, and refresh has its own controller for the same reason.

`scoria_zq_ctrl` therefore:

- holds the ZQCS interval as a **runtime CSR**, so it can be swept;
- raises a demand to the scheduler rather than seizing the bus;
- enforces t_{ZQCS} before releasing the bus back;
- reports a count of issued calibrations, so the interval can be confirmed from the host rather than assumed.

The scheduler change is one input, not a rework. `scoria_mem_cmd_scheduler` already arbitrates refresh demand against demand traffic; ZQ adds a second maintenance source with the same shape.

Note: whether ZQCS should ever *preempt* queued demand traffic, or only fill gaps, is deliberately not fixed in this edition. The baseline requirement is that periodic ZQCS is issued at all, with the interval observable and settable. Cleverness about placement is a characterization question, and `scoria TASK-001` already owns the survey of scheduling modes.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 Write Leveling: the Interface, Not the Search

11.1 What write leveling is for

On a fly-by command/address topology, DQS arrives at each DRAM at a different time relative to CK. Write leveling lets the controller discover that skew per device: the DRAM is put into a leveling mode via MR1, the controller drives a DQS edge, and the DRAM reports on its prime DQ

bit whether that edge arrived before or after the CK rising edge. The controller walks its DQS delay until the reported value flips. That flip is the answer.

11.2 The decision: scoria provides the interface, firmware runs the search

This was settled as decision D2. `scoria_wrlvl_ifc` contains **no search loop**.

The reasoning is the family's standing architectural boundary. A delay-line walk is PHY calibration: the step size, the number of taps, their monotonicity and their temperature behaviour are all properties of the PHY, not of DDR3. pumice made the same call and levels its board from firmware over a CSR passthrough, with no hardware state machine — thirteen PHY knobs reached indirectly, the search in a host program, the settled result cached.

The alternative — a hardware FSM — buys independence from a host at the cost of importing PHY-specific search logic into a portable controller, and of a calibration bug being a silicon respin instead of a script edit.

Important: this is a boundary decision, not an efficiency one. A future system that needs autonomous leveling should add an external calibration engine driving this interface, not a search loop inside the controller.

Corroborated by LiteDRAM. A DDR3 LiteDRAM core was generated to check this (`bin/litedram_ddr3_ref.yml`). Its write-leveling support is two CSR fields – `wlevel_en` to enter the mode and `wlevel_strobe` to emit a DQS pulse – and **no state machine whatsoever**: a search for leveling FSM state in the generated core returns nothing. Software enables the mode, pulses the strobe, reads the result and walks the delay.

An independent and widely-deployed controller reaching the same conclusion is worth more than the argument above on its own. It also tells us the *shape* of the interface that works in practice: enable plus strobe, rather than a higher-level “do a leveling pass” command.

11.3 What the block owes the system

Table 3.3: `scoria_wrlvl_ifc` responsibilities

Responsibility	Detail
DFI leveling handshake	drive <code>dfi_phylvl_req_cs_n</code> and observe <code>dfi_phylvl_ack_cs_n</code> , with <code>dfi_phy_wrlvl_cs_n</code> selecting write leveling; per chip select, per DFI v3.1
MR1 path	enter and leave the DRAM's write-leveling mode by MRS to MR1, sequenced against the handshake

Responsibility	Detail
Timing windows	enforce tWLMRD, tWLDQSEN, tWLO and tWLOE as runtime CSRs
Result capture	present the prime DQ bit's reported value to the host, per chip select
Telemetry	*_STATS: attempts, flips observed, current window, and whether a leveling pass has ever completed

11.4 The timing windows

From JESD79-3F:

Table 3.4: Write-leveling timing windows

Parameter	Value	What the controller must do
tWLDQSEN	25 nCK minimum	wait this long after entering leveling mode before driving DQS low / DQS# high; by its expiry the DRAM has applied ODT to those signals
tWLMRD	40 nCK minimum	wait this long before the first DQS pulse. The maximum is controller-dependent — the spec explicitly declines to bound it
tWLO	per speed bin	the DRAM returns the result on the prime DQ bit(s) asynchronously, this long after the DQS edge
tWLOE	per speed bin	DQ output uncertainty, defined to permit mismatch across DQ bits

Note on tWLMRD's open maximum. Because JESD79-3F leaves the maximum to the controller, scoria must *define* one rather than inherit it, and the definition belongs in the CSR: a host that waits forever for a result it will never get is indistinguishable from a broken link. The requirement is a CSR-settable timeout with a distinct status bit, so a leveling attempt that never reports is reported as a timeout and not as a flip that failed to arrive.

Note on tWLOE. It exists to allow mismatch *across DQ bits*, which only matters if the controller samples more than the prime bit. If the implementation samples one bit, tWLOE is inert — and that should be an explicit, documented choice rather than an accident.

11.5 The telemetry requirement, and why it is not optional

The *_STATS surface is a requirement rather than a nicety, and the reason is a lesson paid for on pumice's board: a detector that has never fired is not evidence of anything. pumice's data-integrity CRC checker had never once fired, and proving it worked required deliberately corrupting the analog eye until it reported every beat mismatched.

A leveling interface that reports “converged” without exposing the attempts and the observed flips is the same trap. The requirement is therefore that a host can distinguish four states: never attempted, attempted and converged, attempted and timed out, and attempted with no flip found across the whole window.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

12 Refresh: DDR3 All-Bank and LPDDR3 Per-Bank

12.1 Marking: INHERITED, with a mode select

This chapter changed between v0.1 and v0.2 of this document. v0.1 marked `refresh_ctrl` MODIFIED on the assumption that LPDDR3 per-bank refresh was new work. It is not: **pumice already implements it**, for LPDDR2, and the mechanism LPDDR3 specifies is the same one.

12.2 What pumice already has

`refresh_ctrl` carries the retention interval, all-bank refresh, JEDEC's plus-or-minus-eight postpone and pull-in scheduling — proven in formal across all sixteen postpone values — **and a REFpb bank rotor**.

The rotor is the part worth understanding, because it is not a scheduler:

`r_bank_rotor` advances exactly when a REFpb COMMAND is granted onto the wire, and HOLDS through REFab mode.

It mirrors the **device's** internal bank counter. pumice's own comment cites JESD209-2 §6.6 for why: the REFpb command *carries no bank address*. The controller cannot nominate a bank. It can only track which bank the device will refresh next, so that the bank timers know which bank is about to become busy.

The rotor also deliberately holds across a mode change rather than resetting, because the device's counter persists — and clearing the mirror would desynchronise it from the device.

12.3 LPDDR3 keeps the same mechanism

JESD209-3C specifies a bank counter in the memory device, and states that the bank sequence for per-bank refresh is **fixed to be a sequential round-robin**. During a REFpb operation, banks other than the one being refreshed remain accessible.

That is the LPDDR2 mechanism unchanged, so the pumice implementation transfers.

Important — this retires an open question. v0.1 of this document asked (as Q5) whether the REFpb round-robin order should be strictly sequential or follow bank occupancy. **The question was malformed.** Occupancy-aware ordering is not available at any price: the command carries no bank address and the device's sequence is fixed. There is nothing to choose. The question is struck rather than deferred, because deferring it would imply a decision still exists.

12.4 Refresh by memtype

Table 3.5: Refresh by memtype

Memtype	Refresh available
DDR3	all-bank REF only, with the JEDEC postpone window. DDR3 adds no controller-directed per-bank refresh
LPDDR3	all-bank REF, plus REFpb in the device's fixed sequential order

Per-bank refresh matters because an all-bank refresh blocks every bank at once while REFpb leaves the others accessible. On a workload with bank parallelism that is the difference between a refresh being a stall and being nearly free.

The cleverer schemes — out-of-order per-bank refresh, write-refresh parallelisation, refresh pausing, SARP and DSARP — are LPDDR4/DDR5 or research, and scoria TASK-001 assigns them to the DDR4 area rather than here. That split stands.

12.5 Requirements

1. Offer all-bank refresh with the JEDEC postpone/pull-in window (inherited), and for LPDDR3 additionally REFpb with the rotor mirroring the device's counter (inherited).
2. The all-bank versus per-bank choice on LPDDR3 is a **runtime CSR**, so both can be measured from one bitstream. This is the only genuinely new surface in this block, and it is a mode select rather than a mechanism.
3. **Re-establish the retention property for the per-bank path.** The inherited formal proof covers retention headroom across every postpone value for all-bank refresh. REFpb changes the arithmetic that proof checks — the interval seen by any one bank is the all-bank interval multiplied by the bank count — so the property must be re-derived for the per-bank mode.
4. REFpb must respect per-bank timing as an ordinary occupant of that bank: a per-bank refresh is an operation on a bank and the bank timer owns it.

Important: requirement 3 is the one most likely to be skipped, and it is the reason refresh can be trusted at all. Carrying the all-bank property forward unchanged would prove the wrong thing while appearing green — the failure mode this family has been bitten by before.

Note: pumice's idle detection is inherited with the block and is worth knowing about, because it looks like an arbitrary constant. Demand is CAM occupancy, which blinks off for a few cycles between bursts; treating those micro-gaps as idle would release postponed refreshes mid-stream and trigger pull-ins. Only a sustained gap counts, confirmed over sixteen cycles.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 DFI v3.1 Master Interface, and the v2.1.1 Delta

13.1 The revision decision

scoria targets **DFI v3.1** (decision D1). pumice's DFI layer was built to v2.1.1, so this chapter is the delta between them, measured across the two specifications: 126 distinct `dfi_*` signal names in v2.1.1, 151 in v3.1.

The decision was not obvious and the reasoning is on the record: v3.1's *gain* for a DDR3/LPDDR3 controller is the leveling rework, the low-power split and `dfi_init`. Everything else it adds is DDR4/LPDDR4. Building to v2.1.1 and hand-adding write leveling would reuse pumice's layer more directly. v3.1 wins because the per-CS leveling scheme is what the DDR4 controller will need too, and paying for it once here is cheaper than paying twice.

13.2 What does NOT change, and a diff that lies about it

The datapath is inherited entirely. One async-FIFO CDC, bubble-free, the unit of transfer a DFI word, `DFI_RATE = 4`.

Important. A naive signal-name diff between the two specifications reports `dfi_address_p2`, `dfi_cs_n_p3`, `dfi_bank_n_p2`, `dfi_odt_p3` and their siblings as **removed in v3.1**. They are not removed. v2.1.1 enumerates phase variants explicitly; v3.1 generalises the notation to `_pN` — `dfi_address_pN`, `dfi_cs_n_pN`, `dfi_odt_pN`, `dfi_bank_pN` — and shows `_p0/_p1` only as examples. The 1:4 frequency ratio remains a defined mode (v3.1 §3.2.4, §3.3.4).

pumice runs four phases. An implementer who trusted that diff would rewrite a working, measured datapath for no reason, so the trap is recorded here rather than left for them to fall into.

13.3 What changes and matters

13.3.1 Leveling: reworked outright

Table 4.1: The leveling interface, v2.1.1 against v3.1

v2.1.1	v3.1
<code>dfi_rdlvl_mode</code> , <code>dfi_rdlvl_load</code> , <code>dfi_rdlvl_delay*</code> , <code>dfi_rdlvl_gate_mode</code>	removed
<code>dfi_wrlvl_mode</code> , <code>dfi_wrlvl_load</code> , <code>dfi_wrlvl_delay*</code>	removed
<code>dfi_rdlvl_cs_n</code> , <code>dfi_wrlvl_cs_n</code> —	removed <code>dfi_phylvl_req_cs_n</code> , <code>dfi_phylvl_ack_cs_n</code> (request/ack, per chip select)
—	<code>dfi_phy_wrlvl_cs_n</code> , <code>dfi_phy_rdlvl_cs_n</code> , <code>dfi_phy_rdlvl_gate_cs_n</code> (which leveling)
—	<code>dfi_lvl_pattern</code> , <code>dfi_lvl_periodic</code>

This is the change scoria cannot route around, because DDR3 adds write leveling. `scoria_wrlvl_ifc` (Chapter 3.3) drives this interface, and because the search is firmware-driven the block is an interface and a set of timing windows rather than a state machine.

13.3.2 Low power: one request becomes two

`dfi_lp_req` splits into `dfi_lp_ctrl_req` and `dfi_lp_data_req`, letting the command path and the data path be put to sleep independently. `powerdown_ctrl` drives both.

13.3.3 Initialization and errors

`dfi_init` appears, giving initialization its own interface — which pairs with DDR3's RESET# pin. `dfi_parity_error` becomes `dfi_error` plus `dfi_error_info`, a wider surface that mostly carries DDR4 CA-parity detail; scoria uses the error indication and ignores the DDR4-specific encodings.

13.4 What scoria does NOT implement

Named explicitly so their absence is a decision on the record:

Table 4.2: DFI v3.1 features deliberately unimplemented

Group	Signals	Why not
DDR4 command encoding	<code>dfi_act_n*</code> , <code>dfi_bg*</code> , <code>dfi_cid*</code>	DDR3 has no dedicated ACT pin, no bank groups, no chip ID
CA parity, CRC, alert	<code>dfi_alert_n*</code> , <code>dfi_parity_in_p*</code>	DDR4-era
Data Bus Inversion	<code>dfi_rddata_dbi*</code>	DDR4/LPDDR4
CA training	<code>dfi_calvl_*</code> , <code>dfi_ca_capture</code> , <code>dfi_phy_calvl_cs_n</code>	LPDDR4-era

That is the majority of v3.1's growth over v2.1.1. Targeting the newer revision does not mean implementing all of it.

14 AXI4 Slave and APB CSR

Both interfaces are **INHERITED** from pumice without functional change. This chapter records what that inheritance includes, so that “inherited” is not mistaken for “unspecified”.

14.1 AXI4 slave

The host-side interface is pumice’s `axi4_ifc`: one read intake and one write intake at 1:1, a write-data CAM, a read-reorder CAM, in-order commit, and FSM-free split and aggregate where the aggregate and last flags ride with the command in the CAM.

Nothing about DDR3 or LPDDR3 reaches the host side. The DRAM generation is invisible across this boundary, which is the property that makes the family worth having.

Width gearing is inherited too. The host bus width need not match the DRAM burst width: formally-verified `dwidth` converters wrap the core, so the host side is free. This is why `scoria_top_geared` exists as a separate top alongside `scoria_top`.

14.2 APB CSR

A PeakRDL-generated register block behind an APB slave. The register *set* grows (Chapter 5), but the interface and the generation flow do not change.

Important — the regeneration rule applies unchanged. The register block is generated through `bin/peakrdl_generate.py`, never `raw peakrdl`, and always with an explicit `-o`. The wrapper emits the RTL, the documentation and the Python regmap together; `raw peakrdl` skips the latter two and the default output path can write an orphan copy the build never reads, which then drifts.

Important — registers are accessed by NAME. Host code reads and writes through the generated regmap, never by hardcoded offset. Offsets churn whenever the RDL changes, and a host that hardcodes them fails silently by hitting a neighbouring register that happens to decode.

14.3 The CDC on the CSR path, and why it is an async FIFO

If scoria is integrated the way pumice is on its board — an APB window on one reset domain reaching a core on another — then the CDC between them must be a gray-pointer async FIFO, not a toggle-parity handshake.

The reason is an asymmetric reset, and it is worth carrying forward because it cost real debugging on pumice: the APB side runs on `presetn` and the core side on `aresetn`, and a soft reset pulses **only the core side**. A toggle-parity handshake desynchronises across that and leaves the response stream permanently offset by one transaction — every readback returns the answer to the previous question, which looks like a lagging register file rather than a CDC fault.

This is a note about integration rather than about scoria’s own ports, but it belongs here because the next person to simplify that CDC will be reading this chapter.

15 scoria_pkg, and Build-Time vs Runtime

15.1 The package (decision D3)

scoria gets its own `scoria_pkg`, with a one-bit memtype enum:

```
typedef enum logic {  
    MEMTYPE_DDR3    = 1'b0,  
    MEMTYPE_LPDDR3  = 1'b1  
} memtype_e;
```

It does **not** extend `pumice_pkg`. That package’s `memtype_e` is one bit wide and already spent on {DDR2, LPDDR2}; adding two more members means widening the field, which reaches pumice’s RTL and its CSR — a measured, shipping design — for the benefit of a controller with no RTL. The trade is wrong in that direction.

When this changes. At the start of the DDR4/LPDDR4 controller, a shared `mem_ctrl_pkg` with a two-bit memtype becomes worth the migration, and both existing controllers move to it in the same pass. That is the right moment because the cost is paid once with three members to amortise it.

Note: until then `scoria_pkg` and `pumice_pkg` carry near-identical type definitions. The duplication is deliberate and time-boxed. It is recorded here so that a later reader does not “fix” it by widening pumice’s CSR — which is precisely the thing this decision avoids.

Inherited from pumice’s package: the row field is already 18 bits wide, with the comment “DDR3 forward-compat”. That was written in anticipation of this controller and carries over as-is.

15.2 The rule: timings are runtime, geometry is build-time

Inherited from pumice and restated because it is the single most load-bearing convention in the family:

Every enforced timing is a runtime CSR. Not a parameter. A timing compiled in cannot be swept, and a controller whose timings cannot be swept cannot be characterized — which is how a controller ends up with numbers nobody can explain.

Build-time is reserved for what cannot be runtime: bus widths, the DFI gear ratio, CAM depths, bank and rank counts. The test is whether changing it changes the amount of hardware.

Table 5.1: Build-time versus runtime

Kind	Build-time or runtime	Examples
Bus and geometry	build	data width, address width, NUM_BANKS, NUM_RANKS, CAM depths
DFI gearing	build	DFI_RATE, which must equal the PHY's phase count
Memtype	build	MEMTYPE_DDR3 / MEMTYPE_LPDDR3
JEDEC timings	runtime	every command-spacing and recovery parameter
New DDR3 timings	runtime	tZQinit, tZQoper, tZQCS, tWLMRD, tWLDQSEN, tWLO, tWLOE
Policy selects	runtime	page policy, refresh mode (all-bank vs REFpb), ZQCS interval

Important — DFI_RATE must equal the PHY's phase count, set identically on both sides. This is inherited and it is not negotiable: a mismatch is not a performance problem, it is a functional break, and it is the kind that presents as data corruption rather than as an error.

15.3 New CSR groups

The register set grows by three groups. Exact offsets are not fixed in this edition — they come from the RDL, which is authored with the RTL — but the content is specified:

Table 5.2: New CSR groups

Group	Contents
ZQ	tZQinit, tZQoper, tZQCS, the ZQCS interval, and a count of calibrations issued
Write leveling	tWLMRD (with its controller-defined maximum as a timeout), tWLDQSEN,

Group	Contents
Refresh mode	tWLO, tWLOE, per-CS result, and the *_STATS telemetry of Chapter 3.3 all-bank versus REFpb select, for LPDDR3

Note: tWLMRD's maximum is a *scoria* number, not a JEDEC one — JESD79-3F declares it controller-dependent. It appears in the CSR as a timeout with its own status bit, so that a leveling attempt which never reports is distinguishable from one that reports no flip.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Verification Strategy, and the Open Questions

16.1 The target is the DFI boundary

Verification is against the DV repository's DFI bus functional model, in cocotb. No board (Sean, 2026-09-29: sim now, board decision later).

That boundary is chosen for the same reason pumice chose it: it is the one interface where the controller's obligations are fully specified by a standard, so a model can be authoritative rather than approximate. pumice's experience is that code passing against the BFM passes on hardware modulo PHY training — which is exactly the residue this specification pushes into firmware anyway.

16.2 What must be verified, beyond inheriting pumice's suites

The inherited blocks bring their tests with them. The new and modified ones need new coverage, and four items are specific enough to name now:

1. **The init sequence, in JEDEC's order.** Not “the DRAM came up” but step-by-step: RESET# held, the 500 us wait, clocks stable before CKE, CKE continuously high, tXPR before the first MRS, then MR2, MR3, MR1, MR0, then ZQCL, then tDLLK and tZQinit. A checker that asserts the *order*, because order is what pumice got wrong on its equivalent.
2. **ODT during init.** High-impedance while RESET# is asserted, and held statically — LOW if RTT_NOM is enabled in MR1 — until init completes.

3. **Write leveling as a protocol, not an outcome.** The handshake sequence and every timing window, with the `twLMRD` timeout exercised deliberately. A leveling interface whose timeout path has never run is an untested path on the only path that reports failure.
4. **Per-bank refresh retention.** The inherited formal property proves retention headroom across all sixteen postpone values for all-bank refresh. `REFpb` changes that arithmetic — the per-bank interval is the all-bank interval times the bank count — so the property must be re-established, not carried forward. Carried forward unchanged it would prove the wrong thing and appear green.

Important — the formal area is the right home for the spacing properties. No assertions go in the RTL. `pumice`'s `formal/pumice` area proved nine modules and found two real bugs, including the `global_timers` next-state defect this specification tells `scoria` to inherit the fix for. `scoria` should open its own formal area early, and the `cmd_history_checker` — which independently re-derives JEDEC spacing from the issued command stream — should grow DDR3's parameters before the RTL is trusted.

16.3 Question resolutions (v0.2)

Five questions were listed in v0.1. Two are answered, one is struck as malformed, and two are deferred with a named condition.

Table 6.1: Question resolutions

#	Question	Resolution
Q1	DFI low-power handshake ordering on exit	DEFERRED (Sean). Depends on PHY behaviour, which is downstream of the board decision. Condition to revisit: a PHY is chosen
Q2	May periodic ZQCS preempt queued demand traffic?	ANSWERED: no. Request/grant, never preempt. See below
Q3	Sample one prime DQ bit or several?	ANSWERED: the prime bit only , which makes <code>twL0E</code> inert. See below

#	Question	Resolution
Q4	The value of tWLMRD's controller-defined maximum	DEFERRED. Needs a PHY-informed number; same condition as Q1
Q5	REFpb ordering: sequential or occupancy-aware?	STRUCK — the question was malformed. See Chapter 3.4

16.3.1 Q2: ZQCS is a request, never a preemption

Answered from a DDR3 LiteDRAM core, generated for this purpose. v0.2 answered this by analogy, because the LiteDRAM core in the pumice flow is a DDR2 configuration and contains no ZQ logic at all. Sean's suggestion was to generate a DDR3 one; that core now exists and the analogy is no longer needed.

What the DDR3 core does (bin/litedram_ddr3_ref.yml regenerates it):

- **ZQCS shares the refresher's FSM.** There is one refresher state machine, and it starts the ZQCS executer from inside itself and waits for its done. Maintenance is not a separate agent competing for the bus; it is one sequencer issuing both kinds of maintenance command.
- **It requests, and waits.** refresh_req to every bank machine is driven from the maintenance command's valid. The bank machines must grant – drain and precharge – before the command issues. Nothing is preempted.
- **The executer is a timed sequence,** a trigger counter stepping to 19 and then asserting done, with the periodic timer held off (zqcs_timer_wait) until it completes. So tZQCS is enforced by holding the *next* interval, not by blocking traffic.

That is three independent controllers agreeing – LiteDRAM DDR2 refresh, LiteDRAM DDR3 ZQCS, and pumice refresh – so the baseline is settled rather than merely plausible: scoria_zq_ctrl raises a demand and waits for a grant.

Design consequence worth taking from LiteDRAM: it shares one sequencer between refresh and ZQ rather than giving ZQ its own. scoria's Chapter 2.3 specifies scoria_zq_ctrl as a separate FUB, and that remains the plan – pumice's refresh_ctrl is already substantial and adding a second maintenance mode to it would couple two independently-verifiable things. But the *interface* both present to the scheduler should be identical, so that the arbiter sees one kind of maintenance demand with a source tag rather than two special cases.

Condition to revisit unchanged: if characterization shows ZQCS starved under sustained demand – the interval slipping materially past its CSR value – placement becomes TASK-001's

scheduling question. The issued-calibration counter required in Chapter 3.2 is what makes that measurable rather than suspected.

16.3.2 Q3: sample the prime DQ bit only

The write-leveling interface samples the prime DQ bit and not the full byte. The consequence is explicit: **twLOE is inert in this implementation**. It exists in JESD79-3F to allow mismatch *across* DQ bits, which only matters to a controller sampling several.

twLOE remains a CSR field so that a later implementation can sample wider without a register-map change, and it is documented as unused rather than omitted — an absent register is a silent limitation, while a present one documented as inert is a decision a reader can find.

16.3.3 Q5: struck rather than deferred

v0.1 asked whether REFpb round-robin should be sequential or occupancy-aware. There is nothing to choose: the REFpb command carries no bank address, the sequence is fixed in the device, and the controller's rotor only mirrors the device's counter. JESD209-3C states the sequence is fixed sequential round-robin; pumice's implementation already cites JESD209-2 §6.6 for the same property in LPDDR2.

It is struck rather than deferred because deferring implies a decision still exists. This one never did, and the correction also moved `refresh_ctrl` from MODIFIED to INHERITED — see Chapter 3.4.

16.4 What would make this a 1.0

Three things, in order:

1. The two deferred questions (Q1, Q4) answered — both blocked on a PHY choice, which is downstream of the board decision.
2. The RTL written, and this document reconciled against it — with every INHERITED marking either confirmed or corrected. A marking that turns out to be wrong is a defect in this document, and the correction goes here rather than being dropped silently.
3. The exact CSR map, which comes from the RDL and cannot honestly precede it.

Until then this is a 0.1: a specification good enough to implement against, and explicit about where it is not yet a description of anything.